COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09 - 09 - 26	**DATE SUBMITTED:**	09 - 16 - 26
NAME:	Paula Esguerra	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA NO. _8__ Sorting Algorithms

1. PROCEDURE OUTPUT

Array of Values for Sort Algorithm Testing

Screenshot

main.cpp

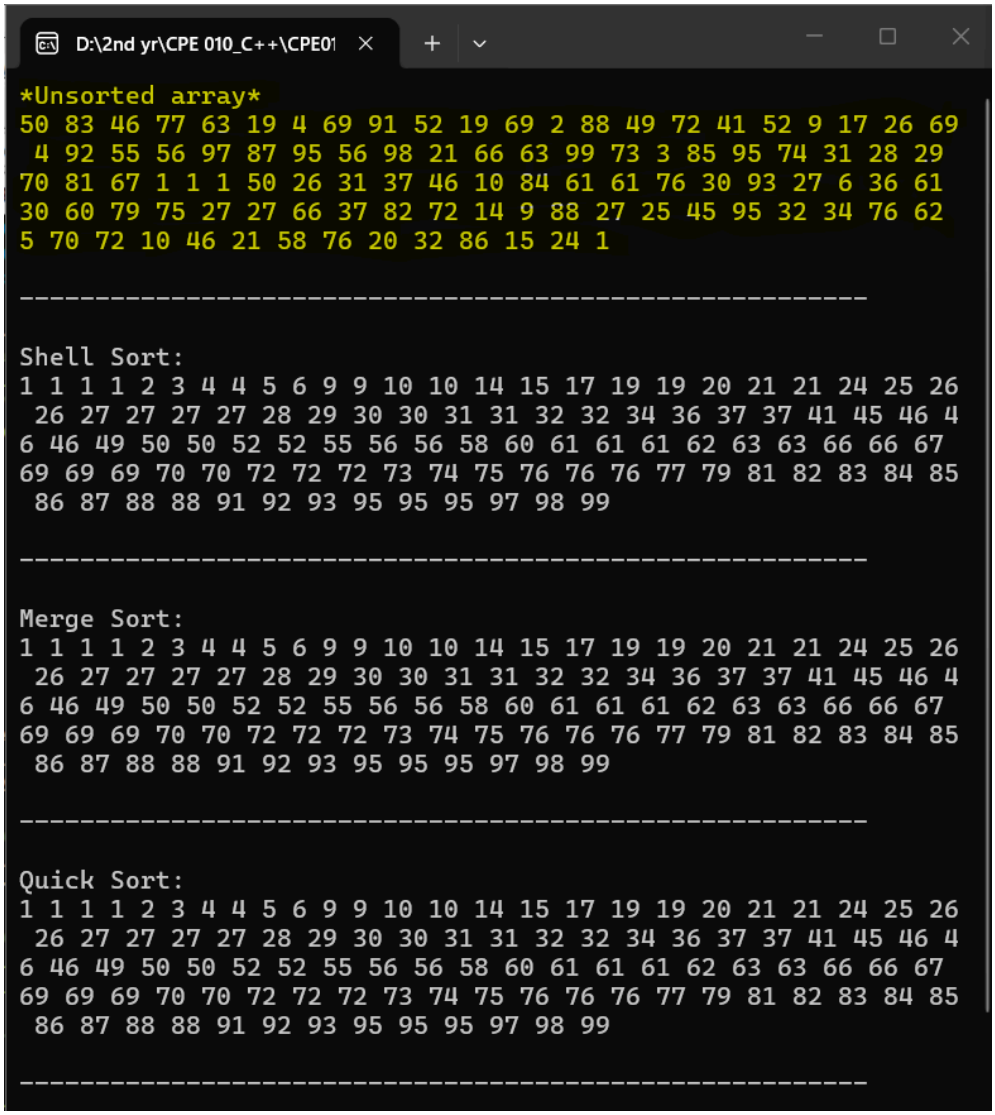
```
    // Create an array with unsorted list with 100 maximum size
    int unsorted_arr[max_size] = {};

    // Use time (0) as seed for random values
    std::srand(std::time(0));

    // Populate the array
    for (int i = 0; i < max_size; i++) {
        unsorted_arr[i] = std::rand() % 100;
    }

    // Display the unsorted array
    std::cout << "*Unsorted array*" << std::endl;
    displayArr(unsorted_arr);
    std::cout << std::endl;
    std::cout << "----------------------------------------------------------------" << std::endl;
    std::cout << std::endl;
```

Output



Observations

I got this from the last activity, where the array is randomly generated, then sorted through the different sorting techniques. I added just some designs to keep it organized and so that it would be easy to differentiate.

Shell Sort Technique	

Code	**main.cpp**
	`std::cout << "Shell Sort: " <<std::endl;`
	`shellSort(unsorted_arr, max_size);`
	`displayArr(unsorted_arr);`
	`std::cout << std::endl;`
	`std::cout << "----------" << std::endl;`
	`std::cout << std::endl;`
	
	sortingAlgo_main.h
	`void shellSort(int arr[], int n) {`
	`  // Start with a big gap, then reduce it`
	`  for (int gap = n / 2; gap > 0; gap /= 2) {`
	`  // Perform a gapped insertion sort`
	`  for (int i = gap; i < n; i++) {`
	`  int temp = arr[i];`
	`  int j = i;`
	`  while (j >= gap && arr[j - gap] > temp) {`
	`  arr[j] = arr[j - gap]; //shift elements forward`
	`  j -= gap;`
	`  }`
	`  arr[j] = temp;`
	`  }`
	`  }`
	`}`

Output



Observations

Shell Sort is a comparison sort of a kind, like a insertion sort. It allows the exchange of far apart elements, speeding up the process by shifting elements large steps toward their final position.

Merge Sort Algorithm

Code

main.cpp

```
37
38      std::cout << "Merge Sort: " <<std::endl;
39      shellSort(unsorted_arr, max_size);
40      displayArr(unsorted_arr);
41      std::cout << std::endl;
42      std::cout << "------------------------------------------------------------" << std::endl;
43      std::cout << std::endl;
```

sortingAlgo_main.h

```
20
21  void merge (int arr[], int low, int mid, int high){
22      int n1 = mid - low + 1;
23      int n2 = high - mid;
24
25      //temporary array
26      int* leftArr = new int[n1];
27      int* rightArr = new int[n2];
28
29      // copying data to temporary array
30      for (int i = 0; i < n1; i++) leftArr[i] = arr[low + i];
31      for (int j = 0; j < n2; j++) rightArr[j] = arr[mid + 1 + j];
32
33      // Merge the temporary arrays back into arr
34      int i = 0, j = 0, k = low; // three pointers
35      while (i < n1 && j < n2) { // items that compares the array until the last one
36          if (leftArr[i] <= rightArr[j]) { //sorting decision left to the right
37              arr[k] = leftArr[i]; // if the left item is smaller or equal
38              i++;
39          } else { // if the right item is smaller, it moves one step
40              arr[k] = rightArr[j];
41              j++;
42          }
43          k++;
44      }
45
46      // Copy any remaining elements of leftArr[]
47      while (i < n1) {
48          arr[k] = leftArr[i];
49          i++;
50          k++;
51      }
52
53      // Copy any remaining elements of rightArr[]
54      while (j < n2) {
```

```
55              arr[k] = rightArr[j];
56              j++;
57              k++;
58          }
59
60          // Clean up memory to avoid memory leaks
61          delete[] leftArr;
62          delete[] rightArr;
63      }
64
65      void mergeSort(int arr[], int low, int high) {
66
67          if(low >= high) return;
68
69          if (low < high) {
70              int mid = low + (high - low) / 2;
71              mergeSort(arr, low, mid);
72              mergeSort(arr, mid + 1, high);
73
74              merge(arr, low, mid, high);
75          }
76      }
77
```

Output

```
*Unsorted array*
50 83 46 77 63 19 4 69 91 52 19 69 2 88 49 72 41 52 9 17 26 69
 4 92 55 56 97 87 95 56 98 21 66 63 99 73 3 85 95 74 31 28 29
70 81 67 1 1 1 50 26 31 37 46 10 84 61 61 76 30 93 27 6 36 61
30 60 79 75 27 27 66 37 82 72 14 9 88 27 25 45 95 32 34 76 62
5 70 72 10 46 21 58 76 20 32 86 15 24 1

------------------------------------------------------------

Shell Sort:
1 1 1 1 2 3 4 4 5 6 9 9 10 10 14 15 17 19 19 20 21 21 24 25 26
 26 27 27 27 27 28 29 30 30 31 31 32 32 34 36 37 37 41 45 46 4
6 46 49 50 50 52 52 55 56 56 58 60 61 61 61 62 63 63 66 66 67
69 69 69 70 70 72 72 72 73 74 75 76 76 76 77 79 81 82 83 84 85
 86 87 88 88 91 92 93 95 95 95 97 98 99

------------------------------------------------------------

Merge Sort:
1 1 1 1 2 3 4 4 5 6 9 9 10 10 14 15 17 19 19 20 21 21 24 25 26
 26 27 27 27 27 28 29 30 30 31 31 32 32 34 36 37 37 41 45 46 4
6 46 49 50 50 52 52 55 56 56 58 60 61 61 61 62 63 63 66 66 67
69 69 69 70 70 72 72 72 73 74 75 76 76 76 77 79 81 82 83 84 85
 86 87 88 88 91 92 93 95 95 95 97 98 99

------------------------------------------------------------

Quick Sort:
1 1 1 1 2 3 4 4 5 6 9 9 10 10 14 15 17 19 19 20 21 21 24 25 26
 26 27 27 27 27 28 29 30 30 31 31 32 32 34 36 37 37 41 45 46 4
6 46 49 50 50 52 52 55 56 56 58 60 61 61 61 62 63 63 66 66 67
69 69 69 70 70 72 72 72 73 74 75 76 76 76 77 79 81 82 83 84 85
 86 87 88 88 91 92 93 95 95 95 97 98 99

------------------------------------------------------------
```

Observations

It uses a divide and conquer where it splits the array in half until single elements remain, then it sorts then out in order. It does not depend on the initial ordering of the elements.

Quick Sort Algorithm	

Code	**main.cpp**
	`std::cout << "Quick Sort: " <<std::endl;`
	`quickSort(unsorted_arr, 0, max_size - 1);`
	`displayArr(unsorted_arr);`
	`std::cout << std::endl;`
	`std::cout << "----------" << std::endl;`
	`std::cout << std::endl;`
	
	sortingAlgo_main.h
	`void swap(int& a, int& b) {`
	`  int temp = a;`
	`  a = b;`
	`  b = temp;`
	`}`
	
	`int partition(int arr[], int low, int high){`
	`  int Mid = arr[high];`
	`  int i = low - 1;`
	
	`  for (int j = low; j < high; j++){`
	`  if (arr[j] <= Mid){`
	`  i++;`
	`  swap(arr[i],arr[j]);`
	`  }`
	`  }`
	`  swap(arr[i + 1], arr[high]);`
	`  return i + 1;`
	`}`
	
	`void quickSort(int arr[], int low, int high) {`
	`  if(low < high){`
	`  int mid = partition(arr, low, high);`
	
	`  quickSort(arr, low, mid - 1);`
	`  quickSort(arr, mid + 1, high);`
	`  }`
	`}`

Output

```
*Unsorted array*
50 83 46 77 63 19 4 69 91 52 19 69 2 88 49 72 41 52 9 17 26 69
 4 92 55 56 97 87 95 56 98 21 66 63 99 73 3 85 95 74 31 28 29
70 81 67 1 1 1 50 26 31 37 46 10 84 61 61 76 30 93 27 6 36 61
30 60 79 75 27 27 66 37 82 72 14 9 88 27 25 45 95 32 34 76 62
5 70 72 10 46 21 58 76 20 32 86 15 24 1

------------------------------------------------------------

Shell Sort:
1 1 1 1 2 3 4 4 5 6 9 9 10 10 14 15 17 19 19 20 21 21 24 25 26
 26 27 27 27 27 28 29 30 30 31 31 32 32 34 36 37 37 41 45 46 4
6 46 49 50 50 52 52 55 56 56 58 60 61 61 61 62 63 63 66 66 67
69 69 69 70 70 72 72 72 73 74 75 76 76 76 77 79 81 82 83 84 85
 86 87 88 88 91 92 93 95 95 95 97 98 99

------------------------------------------------------------

Merge Sort:
1 1 1 1 2 3 4 4 5 6 9 9 10 10 14 15 17 19 19 20 21 21 24 25 26
 26 27 27 27 27 28 29 30 30 31 31 32 32 34 36 37 37 41 45 46 4
6 46 49 50 50 52 52 55 56 56 58 60 61 61 61 62 63 63 66 66 67
69 69 69 70 70 72 72 72 73 74 75 76 76 76 77 79 81 82 83 84 85
 86 87 88 88 91 92 93 95 95 95 97 98 99

------------------------------------------------------------

Quick Sort:
1 1 1 1 2 3 4 4 5 6 9 9 10 10 14 15 17 19 19 20 21 21 24 25 26
 26 27 27 27 27 28 29 30 30 31 31 32 32 34 36 37 37 41 45 46 4
6 46 49 50 50 52 52 55 56 56 58 60 61 61 61 62 63 63 66 66 67
69 69 69 70 70 72 72 72 73 74 75 76 76 76 77 79 81 82 83 84 85
 86 87 88 88 91 92 93 95 95 95 97 98 99

------------------------------------------------------------
```

Observations

It also uses divide and conquer like merge sort, but instead of blindly idividing the array in half, it picks a pivot and partitions the elements around it, then recursively sorts the independent sub-lists.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.
Yes, One of the perks properties of quicksort. After partitioning, the pivot is on the final position, and every element on the left is less than the pivot and on the right side is greater than the pivot. Since two sub lists are independent, it does not require to move in the pivot again. For example this array of number [38, 27, 43, 3, 9, 82, 10], pivot is equal to ten, the last element. 3 is less than 10, so it will be swap forward. 9 is less than 10, swap forward. So the array looks like this, 3, 9, 10, 38, 27, 82, 43. So now the two halves are [3, 9] and [38, 27, 82, 43] with 10 locked in between. Sorting the left half using insertion sort: [3,9] is ordered. Using Merge Sort in [38, 27, 82, 43], it switches [38, 27] to [27, 38] and [82, 43] to [43, 82], then merge the two ordered pair, it looks like this [27, 38, 43, 82]. In final look, [3, 9, 10, 27, 38, 43, 82].
Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?
Insertion sort, does 56 comparisons on average, there will no recursive calls, no extra memory and has good behaviour too. Merge sort needs about 59 comparisons but it need recursive calls and copying a temporary array, thile quick sort has partition on bookkeeping on the same tiny input. They have the same O(N • log N) because merge and quick sort both use divide and conquer, they have the same recurrence.

Conclusion:
Insertion sort outperforms advanced algorithms on small arrays because it skips recursive calls, partitioning, and memory allocation overhead. Meanwhile, Merge sort and Quick sort leverage a divide-and-conquer strategy to handle larger datasets efficiently. Consequently, hybrid implementations that combine these distinct methods are highly recommended to achieve maximum speed and optimal resource management. By dynamically switching algorithms based on sub-array size, modern systems ensure the fastest possible execution across any data scale. This adaptive approach balances low-overhead simplicity with high-powered scaling to deliver truly optimized sorting routines.

3vb . ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of my idea of the code**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The idea is original but it need to be double check for correct idea and thinking.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.